

Soutenance de Projet : Base de Données & Java

ARTCONNECT

Plateforme Communautaire d'Art Local

 Hugo Thouraud de Lavignère  Julia Tea  Ines Toure  Kogulaan Vimalan



L'ART AU CŒUR DU NUMÉRIQUE

🎯 Vision & Objectifs

- ✓ **Vision ArtConnect**
Connecter les artistes locaux et les passionnés via une plateforme numérique intuitive.
- ✓ **Objectif Majeur**
Transition d'une gestion volatile en mémoire vers une architecture de données MySQL robuste.
- ✓ **Scalabilité**
Garantir la pérennité des données et la montée en charge de l'écosystème artistique.

📦 Stack Technologique



Java 17 & JavaFX



MySQL 8.0



JDBC Driver



Maven Build

DE LA MÉMOIRE À LA PERSISTANCE

État Initial

- ✗ Données volatiles en mémoire (ArrayList)
- ✗ Perte des données à l'arrêt de l'application
- ✗ Relations complexes difficiles à maintenir
- ✗ Absence d'intégrité référentielle native



Solution Cible

- ✓ Persistance durable via MySQL 8.0
- ✓ Architecture JDBC performante et sécurisée
- ✓ Modèle relationnel normalisé (3FN)
- ✓ Gestion native des contraintes et triggers

Mode Dual : InMemory (Dev) ↔ JDBC (Prod)

EXPÉRIENCE UTILISATEUR & FONCTIONNALITÉS

Hub Artistique

Gestion centralisée des profils artistes
Catalogues d'œuvres avec détails & prix
Annuaire des galeries partenaires

Interactivité

Réservation de workshops en temps réel
Gestion des expositions et thématiques
Espace communautaire et avis

Interface Moderne

Navigation fluide par onglets (JavaFX)
Affichage dynamique des contenus
Tableaux de bord clairs pour lecture

Filtres Avancés

Recherche multicritères par discipline
Tri par prix, statut et localisation
Mise à jour instantanée via JDBC

ARCHITECTURE LOGICIELLE EN COUCHES

PRÉSENTATION (UI)

JavaFX FXML + Controllers



SERVICES

Logique Métier & Orchestration



DAO (DATA ACCESS)

Abstraction JDBC & Mapping



MYSQL DATABASE

Schéma Relationnel & Persistance

Découplage UI

L'interface graphique est **totale**ment isolée de la base de données. Elle communique uniquement avec la couche Service via des interfaces.

Abstraction DAO

La couche DAO encapsule la complexité SQL. Elle utilise **PreparedStatement** pour la sécurité et gère le mapping entre les lignes SQL et les objets Java.

Intégrité & Sécurité

La logique de validation est répartie entre les Services (métier) et MySQL (triggers/contraintes), assurant une **double protection** des données.

MODÉLISATION : UN SCHÉMA RELATIONNEL OPTIMISÉ

Entités Clés

 Artist	 Artwork
 Gallery	 Exhibition
 Workshop	 CommunityMember
 Review	

Relations & Cardinalités

Many-to-Many (N:M)

Gérées via tables d'association pour garder la flexibilité.

artist_discipline	exhibition_artwork
member_favorite	

Intégrité Référentielle

Utilisation de `ON DELETE CASCADE` pour conserver la cohérence lors des suppressions.

Normalisation 3FN

1FN : Atomicité

Attributs atomiques et non-agrégés.

2FN : Dépendance Totale

Suppression des dépendances partielles à la clé.

3FN : Dépendance Transitive

Élimination des dépendances transitives entre attributs non-clés.

INTELLIGENCE SQL : VUES & TRIGGERS

🗄 Vues SQL

Abstraction de la complexité relationnelle pour simplifier les lectures applicatives.

```
CREATE VIEW view_artwork_catalog AS
SELECT aw.title, a.name AS artist
FROM artwork aw
JOIN artist a ON aw.artist_id = a.id;
```

- 🛡 Sécurité : Masquage des colonnes sensibles
- ⚡ Performance : Requêtes pré-optimisées
- 📦 Abstraction : Simplification du code Java

⚡ Triggers Métier

Automatisation de l'intégrité des données directement au niveau du moteur MySQL.

```
BEFORE INSERT ON booking
FOR EACH ROW BEGIN
IF current >= max THEN
SIGNAL SQLSTATE '45000';
END IF;
```

- ✅ Anti-Overbooking : Validation automatique
- 📅 Cohérence : Dates d'expositions vérifiées
- 🤖 Autonomie : Logique indépendante de l'UI

🔍 Optimisation

Indexation stratégique pour garantir des temps de réponse constants.

```
CREATE INDEX idx_artwork_price
ON artwork(price);
CREATE INDEX idx_artist_name
ON artist(name);
```

- 🌐 Accélération des recherches filtrées
- ⚙ Réduction de la charge serveur
- 📈 Scalabilité : Supporte de gros volumes

JDBC : LE PONT TECHNIQUE JAVA-MYSQL

Sécurité & Connexion

Gestion des Connexions

Utilisation de `ConnectionManager` pour centraliser l'accès à MySQL via le driver JDBC.

PreparedStatement

Protection native contre les `injections SQL` grâce à la pré-compilation des requêtes et au typage des paramètres.

`DataAccessException`

Encapsulation des erreurs SQL techniques pour une remontée propre vers l'interface utilisateur.

Mapping Objet-Relationnel

Flux Bidirectionnel

Transformation des `ResultSet` en objets Java et persistance des entités via des requêtes paramétrées.

```
// Exemple de mapping DAO
Artist a = new Artist();
a.setName(rs.getString("name"));
a.setBio(rs.getString("bio"));
// Enregistrement dans le registre
registry.register(a, rs.getInt("id"));
```

Abstraction DAO

Interfaces génériques permettant de changer l'implémentation sans impacter les services métier.

LE DÉFI "OOP-FIRST" : IDENTITYREGISTRY

</> Contrainte Design

Les modèles Java (POJO) comme `Artist` ou `Artwork` ne possèdent aucun champ `id`.

Cette approche garantit un code métier pur, totalement indépendant des préoccupations de persistance, mais crée un défi pour le mapping SQL.

```
public class Artist {  
    private String name;  
    private String bio;  
    // AUCUN champ ID ici  
}
```

🔗 Solution : IdentityRegistry

Utilisation d'une `IdentityHashMap` pour associer les références d'objets Java à leurs clés primaires SQL.



SELECT : Enregistrement automatique lors du chargement des données.



SAVE : Mémorisation de l'ID généré par la base de données.



UPDATE/DELETE : Récupération transparente de l'ID via la référence objet.

TRANSACTIONS ET ROBUSTESSE DES DONNÉES

Principes ACID

A

Atomicité

L'opération est indivisible : tout est validé (Commit) ou tout est annulé (Rollback).

C

Cohérence

La base passe d'un état valide à un autre, respectant toutes les contraintes métier.

I

Isolation

Les transactions s'exécutent sans interférer les unes avec les autres.

D

Durabilité

Une fois validées, les données sont enregistrées de façon permanente sur le disque.

Cas d'Usage : Réservation



Début Transaction : `setAutoCommit(false)`



Vérification de l'existence du membre et de l'atelier



INSERT INTO booking : Déclenchement du Trigger SQL



Si erreur (ex: overbooking) → **Rollback**



Si succès → **Commit** & Fermeture Connexion

DÉMONSTRATION TECHNIQUE & RÉSULTATS

Performance & Stabilité

Réactivité Optimale

Temps de réponse instantanés grâce à l'indexation stratégique et à l'optimisation des requêtes JDBC.

Intégrité Garantie

Zéro incohérence de données grâce aux transactions ACID et aux triggers de validation MySQL.

Rendu Applicatif

Interface JavaFX

Navigation fluide par onglets thématiques avec chargement dynamique des données MySQL.

Recherche & Filtres

Exploitation efficace des vues SQL pour des recherches multicritères performantes.



Validation réussie des 20 points techniques du cahier des charges.

BILAN DU PROJET

- ✓ Maîtrise de l'architecture JDBC
- ✓ Modélisation relationnelle 3FN
- ✓ Intégrité métier via Triggers SQL

PERSPECTIVES

- 🚀 Intégration d'un Pool HikariCP
- 🚀 Extension du CRUD complet en UI
- 🚀 Migration vers un ORM (Hibernate)

Merci de votre attention

Questions & Réponses

ArtConnect Pro v1.0 • Projet BDD & Java • EFREI 2026